

Construcción y cambio de base del tensor de polinomios con SymPy y NumPy

Planteamiento

Se consideraron

$$P_2(x) = \text{span}\{1, x, x^2\}, \quad G_2(y) = \text{span}\{1, y, y^2\},$$

y su producto tensorial

$$T_2(xy) = P_2(x) \otimes G_2(y).$$

Cualquier polinomio en dos variables dentro de T_2 se expresó como

$$T(x, y) = \sum_{i,j} c^{ij} e_i^P(x) e_j^G(y),$$

donde $\{e_i^P\}$ y $\{e_j^G\}$ son bases de P_2 y G_2 , respectivamente, y c^{ij} son las componentes del tensor respecto a la base producto.

En el problema se usaron los polinomios

$$p^P(x) = x^2 + x + 3, \quad p^G(y) = y + 1,$$

y se pidió:

- (a)–(b) Obtener c^{ij} en la base monomial $\{1, x, x^2\} \otimes \{1, y, y^2\}$.
- (c) Expandir $p^P(x)$ en la base de Legendre $\{P_0, P_1, P_2\}$.
- (d) Obtener $\tilde{c}^{ij}$ en la base de Legendre para x e y .

Lógica computacional

Se trabajó con tres rutinas centrales.

1. Conversión de coordenadas en una base univariante

Dado un polinomio $q(\xi)$ y una base $\mathcal{B} = \{b_0(\xi), \dots, b_n(\xi)\}$, se buscó el vector de coordenadas $\mathbf{a}$ tal que

$$q(\xi) = \sum_{\ell=0}^n a_{\ell} b_{\ell}(\xi).$$

Para ello, se construyó la matriz de cambio de base $M \in \mathbb{R}^{(n+1) \times (n+1)}$ cuyas columnas son los coeficientes monomiales de cada $b_{\ell}(\xi)$:

$$M_{k\ell} = [\xi^k] b_{\ell}(\xi), \quad k, \ell = 0, \dots, n.$$

El vector del lado derecho $\mathbf{b}$ se formó con los coeficientes monomiales de $q(\xi)$:

$$b_k = [\xi^k] q(\xi).$$

Las coordenadas se obtuvieron resolviendo el sistema lineal

$$M \mathbf{a} = \mathbf{b} \implies \mathbf{a} = M^{-1} \mathbf{b}.$$

En el código, esta lógica se implementó en la función `coordenadas_en_base` mediante `LUsolve` de SymPy.

2. Producto tensorial de vectores de coeficientes

Dados dos vectores de coordenadas univariantes $\mathbf{a} = (a_0, \dots, a_n)^{\top}$ para x y $\mathbf{b} = (b_0, \dots, b_m)^{\top}$ para y , las componentes del tensor en la base producto se obtuvieron con el producto exterior:

$$c^{ij} = a_i b_j, \quad i = 0, \dots, n, \quad j = 0, \dots, m.$$

En el código, esto se realizó con la función `producto_exterior`, que devolvió una matriz $C = (c^{ij})$.

3. Reconstrucción del polinomio bivariante

Con C y las bases $\{e_i^P(x)\}$, $\{e_j^G(y)\}$, se reconstruyó

$$T(x, y) = \sum_{i,j} c^{ij} e_i^P(x) e_j^G(y).$$

En el código, esta combinación se implementó en `tensor_desde_coeficientes` y se verificó simbólicamente que coincidía con la expansión directa.

Resultados

(a)–(b) Base monomial

Se expresaron $p^P(x)$ y $p^G(y)$ en las bases monomiales $\{1, x, x^2\}$ y $\{1, y, y^2\}$, respectivamente.

$$\mathbf{a}_{\text{mon}} = [3, 1, 1]^{\top}, \quad \mathbf{b}_{\text{mon}} = [1, 1, 0]^{\top}.$$

El producto exterior produjo

$$C_{\text{mon}} = \begin{pmatrix} 3 & 3 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix},$$

y la reconstrucción dio

$$p^{P \otimes G}(x, y) = (x^2 + x + 3)(y + 1) = x^2y + xy + 3y + x^2 + x + 3.$$

(c) Expansión en la base de Legendre (variable x)

Se usaron $P_0(x) = 1$, $P_1(x) = x$, $P_2(x) = \frac{1}{2}(3x^2 - 1)$. Las coordenadas de $p^P(x)$ resultaron

$$\mathbf{a}_{\text{Leg}} = \left[\frac{10}{3}, 1, \frac{2}{3} \right]^\top,$$

de modo que

$$p^P(x) = \frac{10}{3} P_0(x) + 1 \cdot P_1(x) + \frac{2}{3} P_2(x).$$

(d) Base de Legendre en x e y

Para $p^G(y) = y + 1$ se obtuvo

$$\mathbf{b}_{\text{Leg}} = [1, 1, 0]^\top.$$

El producto exterior entregó

$$\tilde{C} = \begin{pmatrix} \frac{10}{3} & \frac{10}{3} & 0 \\ 1 & 1 & 0 \\ \frac{2}{3} & \frac{2}{3} & 0 \end{pmatrix}.$$

La reconstrucción con $\{P_0, P_1, P_2\}$ en x e y coincidió exactamente con $p^{P \otimes G}(x, y)$, lo que verificó la corrección del cambio de base.